Test Scenarios: B1 Track B Version-Aware RAG

30 copy-paste test cases with expected messages: setup, data, Chroma, ablation, LLM answers, negative tests, scenarios

Modern AI Pro · Level 2 · Track B · RAG Design · Owner: Shreenath Shetty

Version 1 · 2026-10-18 · verified: 30/30 PASS from a fresh GitHub clone (zsh and bash)

What this is	30 copy-paste test cases that prove the B1 system works, from a clean clone of the public GitHub repo
Repo	https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware (branches `rag-deployment`, `track-b-kafka-version-aware`)
Works in	macOS Terminal (zsh) or bash. Everything happens in `~/b1-test`, which is deleted at the end
Time	About 8 minutes for all 30 (TC-30 alone takes about 4)
Verified	2026-10-18: all 30 PASS from a fresh GitHub clone (bash); shell-heavy cases re-verified in zsh

1. How to use this document

1. Open **Terminal**.
2. For each test case, copy the whole grey **Commands** block, paste it, and press Enter.
3. Compare what you see with **Expected message**. Small differences in timings (ms, seconds) are normal.
4. Tick **Pass** or **Fail** in the test record (section 4).
5. Run the cases **in order**. Later cases use what earlier ones built (see **Pre-condition**).

Automatic alternative: the same 30 cases run unattended with one command, which checks every expected message for you:

```
git clone -q -b track-b-kafka-version-aware https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-
python3 /tmp/b1-tests-src/b1-track-b/tools/tests/run_tests.py --shell zsh      # add --skip-llm if you have no
```

Expect the last line `30 PASS · 0 WARN · 0 FAIL · 0 SKIP`. LLM-worded answers are WARN, not FAIL, if the wording differs.

2. Test matrix

ID	Area	Test	LLM / key	Time	Key expected result
TC-01	Setup	Prerequisites are installed	—	< 1 min	versions of git, uv, Python
TC-02	Setup	Clone the deployment branch from GitHub	—	< 1 min	clone OK, run.sh present
TC-03	Setup	Create the Python environment	—	~1–2 min	setup OK · torch 2.2.2
TC-04	Data	Corpus, manifest and golden set are consistent	—	< 1 min	PASS (16 runbooks, 8 docs, 21 cases)

ID	Area	Test	LLM / key	Time	Key expected result
TC-05	Data	The similarity trap is real	—	< 1 min	similarity 0.867–0.969; old wins 4/8
TC-06	Data	Frozen files have the published fingerprints	—	< 1 min	sha1 850c67f2… / dea81e27…
TC-07	Chroma	Ingest the runbooks into Chroma	—	< 1 min	vectors stored 17
TC-08	Chroma	Every chunk in Chroma carries its version	—	< 1 min	9 vs 8 chunks · PASS
TC-09	Chroma	Raw Chroma query, no project scripts	—	< 1 min	chunks 17 · current 9 · superseded 8
TC-10	Chroma	See the filter's effect inside Chroma	—	< 1 min	OFF: 2025 file #2; ON: 0 superseded
TC-11	Ablation	BEFORE: filter OFF	—	< 1 min	48.8% · 0/9 · 7/7 · 0/3
TC-12	Ablation	AFTER: filter ON	—	< 1 min	0.0% · 9/9 · 7/7 · 3/3
TC-13	Ablation	Case by case: what the filter fixed and broke	—	< 1 min	fixed 12 · broken 0
TC-14	Ablation	Results are deterministic and match the committed ones	—	< 1 min	3 × True
TC-15	Ablation	The filter adds no meaningful latency	—	< 1 min	delta < 10 ms · PASS
TC-16	Ablation	Ablation summary table	—	< 1 min	48.8% → 0.0% table
TC-17	Answers (LLM)	Provide the API key (.env)	yes	< 1 min	.env present, ignored by git
TC-18	Answers (LLM)	Stale-threshold question, filter ON	yes	< 1 min	answer 85%, no 2025 file
TC-19	Answers (LLM)	Same question, filter OFF (compare)	yes	< 1 min	2025 file retrieved
TC-20	Answers (LLM)	Exact value question	yes	< 1 min	8085, never 8081
TC-21	Answers (LLM)	Renamed on-call service	yes	< 1 min	streaming-platform-prod
TC-22	Answers (LLM)	Unanswerable question	yes	< 1 min	says not documented
TC-23	Negative tests	Broken manifest is caught	—	< 1 min	exit=1 … restored: PASS
TC-24	Negative tests	Broken golden set is caught	—	< 1 min	FAIL (4) … restored: PASS
TC-25	Negative tests	Chroma without metadata is caught	—	< 1 min	exit=1 · FAIL (18)
TC-26	Scenarios	The filter trusts the metadata (wrong flag)	—	< 1 min	swap serves 2025 file; restore fixes
TC-27	Scenarios	Publish a new runbook version	—	< 1 min	only disk-capacity-v4.md served
TC-28	Project branch	The golden set was frozen before any code change	—	< 1 min	freeze before code: True
TC-29	Project branch	No secrets or personal email in the public repo	—	< 1 min	all zero
TC-30	Project branch	Rebuild every step from scratch (optional, ~4 min)	—	~4 min	7 × MATCH · Done

3. Test cases

A. Setup

Pre-condition for this group: None. Start here.

TC-01 · Prerequisites are installed

Purpose: git, uv and Python are available before anything else.

Commands (copy all, paste, Enter):

```
git --version
uv --version
python3 --version
```

Expected message: Three version lines, e.g. `git version 2.x`, `uv 0.x`, `Python 3.x`.

Pass rule: All three commands print a version. If uv is missing, install it from docs.astral.sh/uv.

TC-02 · Clone the deployment branch from GitHub

Purpose: The public repo is reachable and the rag-deployment branch has the runnable package.

Commands (copy all, paste, Enter):

```
rm -rf ~/b1-test && mkdir -p ~/b1-test && cd ~/b1-test
R=https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware.git
git clone -b rag-deployment $R b1-rag
cd b1-rag && git log --oneline -1 && ls
```

Expected message: Last commit `... rag-deployment: runnable version-aware RAG package`; files include `README.md labs requirements.txt run.sh simple_rag`.

Pass rule: Clone succeeds without a login prompt and run.sh is present.

TC-03 · Create the Python environment

Purpose: run.sh builds a Python 3.12 venv with all packages (Intel-Mac pins applied automatically). *(takes ~1–2 min)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && ./run.sh setup
```

Expected message: `setup OK · torch 2.2.2` (on an Intel Mac; newer torch elsewhere). Takes about 1–2 minutes.

Pass rule: The line `setup OK` appears.

B. Data

Pre-condition for this group: TC-02 and TC-03 done.

TC-04 · Corpus, manifest and golden set are consistent

Purpose: The 16 runbooks, the version manifest and the 21 frozen test cases agree with each other.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_check.py --no-trap
```

Expected message: Lines starting `OK` for corpus, manifest and golden set, ending with `PASS`.

Pass rule: Last line is `PASS`; no `FAIL` line.

TC-05 · The similarity trap is real

Purpose: Old and current runbooks are near-duplicates, so similarity alone can't tell them apart.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_check.py 2>/dev/null | grep -E "pair similarity|outranks|^PASS"
```

Expected message: `pair similarity ... min 0.867 max 0.969`, `old version outranks current for 4/8 plain questions`, `PASS`.

Pass rule: Similarity ≥ 0.85 for every pair and the old version wins at least once.

TC-06 · Frozen files have the published fingerprints

Purpose: The golden set and manifest are byte-identical to the frozen versions.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
shasum simple_rag/golden_kafka.jsonl simple_rag/corpus/kafka/manifest.json
```

Expected message: `850c67f2eab21aa1a6afabd481e5c3efccb7e0ba simple_rag/golden_kafka.jsonl` and `dea81e27843ffee677863915fa998b1dfaf5ab9c .../manifest.json`.

Pass rule: Both fingerprints match exactly.

C. Chroma

Pre-condition for this group: TC-03 done. TC-08 to TC-10 also need TC-07 (ingest).

TC-07 · Ingest the runbooks into Chroma

Purpose: All 16 runbooks are chunked, embedded and stored with their version metadata.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
./run.sh ingest 2>/dev/null | grep -E "md · |chunks produced|vectors stored"
```

Expected message: `md · 16 files`, `chunks produced 17 from 16 docs`, `vectors stored 17`.

Pass rule: 17 vectors stored (one runbook is long enough to split into 2 chunks).

TC-08 · Every chunk in Chroma carries its version

Purpose: The metadata the filter needs is really inside the vector store.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && ./run.sh check | tail -7
```

Expected message: `17 chunks, every one carries ...`, `9 chunks from 8 files` / `8 chunks from 8 files`, `PASS`.

Pass rule: Last line `PASS`.

TC-09 · Raw Chroma query, no project scripts

Purpose: Confirm the metadata directly with the chromadb library.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && .venv/bin/python - <<'PY'
import chromadb
col = chromadb.PersistentClient(path="simple_rag/.chroma").get_collection("kafka")
cur = col.get(where={"superseded": False}, include=["metadatas"])
old = col.get(where={"superseded": True}, include=["metadatas"])
print("chunks:", col.count(), "| current:", len(cur["ids"]),
      "| superseded:", len(old["ids"]))
dated = all(m["path"].endswith(("-2024.md", "-2025.md"))
            for m in old["metadatas"])
print("every superseded file is an old -20xx version:", dated)
PY
```

Expected message: `chunks: 17 | current: 9 | superseded: 8` and `... old -20xx version: True`.

Pass rule: Counts match and the second line says True.

TC-10 · See the filter's effect inside Chroma

Purpose: The same question with and without the filter: the stale 2025 runbook disappears.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && ./run.sh inspect 2>/dev/null | sed -n '/question:/,$p'
N=$(./run.sh inspect 2>/dev/null | sed -n '/filter ON/,$p' | grep -c SUPERSEDED)
echo "SUPERSEDED rows with filter ON: $N"
```

Expected message: Filter OFF lists `disk-capacity-2025.md SUPERSEDED` at #2; filter ON lists only `current` files; last line `SUPERSEDED rows with filter ON: 0`.

Pass rule: OFF shows at least one SUPERSEDED row, ON shows none.

D. Ablation

Pre-condition for this group: TC-07 done (ingest). TC-13, TC-14 and TC-16 use the files written by TC-11 and TC-12.

TC-11 · BEFORE: filter OFF

Purpose: Measure the original retriever on the 21 frozen cases.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_eval.py --off --out local_results 2>/dev/null | tail -7
```

Expected message: `superseded chunks retrieved 41/84 = 48.8%`, `superseded_cases 0/9`, `exact_string 7/7`, `multi_hop 0/3`.

Pass rule: Exactly these four numbers (latency varies).

TC-12 · AFTER: filter ON

Purpose: The same 21 cases with the version filter on.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_eval.py --on --out local_results 2>/dev/null | tail -7
```

Expected message: `superseded chunks retrieved 0/84 = 0.0%`, `superseded_cases 9/9`, `exact_string 7/7`, `multi_hop 3/3`.

Pass rule: 0.0% superseded and exact_string still 7/7: **this is the B1 success condition.**

TC-13 · Case by case: what the filter fixed and broke

Purpose: No case gets worse with the filter on.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && .venv/bin/python - <<'PY'
import json
off = json.load(open("local_results/baseline_filter_off.json"))["cases"]
on = json.load(open("local_results/version_filter_on.json"))["cases"]
w = lambda p: "-" if p is None else ("PASS" if p else "FAIL")
for a, b in zip(off, on):
    print(f"{a['id']}  {a['type']:<13} OFF {w(a['pass']):<4} -> ON {w(b['pass'])}")
pairs = list(zip(off, on))
fixed = sum(a["pass"] is False and b["pass"] is True for a, b in pairs)
broken = sum(a["pass"] is True and b["pass"] is False for a, b in pairs)
print("cases fixed by the filter:", fixed)
print("cases broken by the filter:", broken)
PY
```

Expected message: 21 rows; superseded and multi_hop go `FAIL -> PASS`, exact_string stays `PASS -> PASS`; `cases fixed by the filter: 12`, `cases broken by the filter: 0`.

Pass rule: Fixed = 12, broken = 0.

TC-14 · Results are deterministic and match the committed ones

Purpose: Re-running gives the same cases; your run equals the published results.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_eval.py --off --out local_results2 >/dev/null 2>&1
.venv/bin/python - <<'PY'
import json
def key(f):
    cases = json.load(open(f))["cases"]
    return [(c["id"], c["pass"], [h["file"] for h in c["retrieved"]]) for c in cases]
off, on = "baseline_filter_off.json", "version_filter_on.json"
print("re-run identical:", key(f"local_results/{off}") == key(f"local_results2/{off}"))
print("OFF matches committed:", key(f"local_results/{off}") == key(f"simple_rag/results/{off}"))
print("ON matches committed:", key(f"local_results/{on}") == key(f"simple_rag/results/{on}"))
PY
```

Expected message: `re-run identical: True`, `OFF matches committed: True`, `ON matches committed: True`.

Pass rule: All three True.

TC-15 · The filter adds no meaningful latency

Purpose: OFF and ON timed fairly (alternating, same process).

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_eval.py --latency 2 --out local_results >/dev/null 2>&1
.venv/bin/python - <<'PY'
import json
d = json.load(open("local_results/latency.json"))
off, on, delta = d["off"]["median_ms"], d["on"]["median_ms"], d["median_delta_ms"]
print("OFF", off, "ms | ON", on, "ms | delta", delta, "ms")
print("latency check:", "PASS" if abs(delta) < 10 else "FAIL")
PY
```

Expected message: `OFF ~50 ms | ON ~51 ms | delta ~1 ms` (machine-dependent) and `latency check: PASS`.

Pass rule: Difference under 10 ms.

TC-16 · Ablation summary table

Purpose: One table with all metrics, OFF vs ON.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka RAG_TRACE=0
.venv/bin/python simple_rag/b1_eval.py --table --out local_results 2>/dev/null | sed -n '7,13p'
```

Expected message: Markdown table: superseded `48.8% (41/84)` → `0.0% (0/84)`, exact-string `7/7` → `7/7`, multi-hop `0/3` → `3/3`.

Pass rule: Table shows these values.

E. Answers (LLM)

Pre-condition for this group: TC-07 done. TC-18 to TC-22 need TC-17 (.env) and internet access.

TC-17 · Provide the API key (.env)

Purpose: ask needs an OpenAI-compatible key; it stays local and is never committed. *(needs `.env` and internet)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
cp ~/Desktop/AI-Architect_L2_Aug_2026/project_10082026/.env .env && chmod 600 .env
grep -c "^OPENAI_API_KEY=" .env
git status --short .env; echo "git sees .env: $(git status --short .env | wc -l | tr -d ' ')"
```

Expected message: `1` (one key line) and `git sees .env: 0` (gitignored). The key itself is never printed.

Pass rule: Key present, git ignores the file.

TC-18 · Stale-threshold question, filter ON

Purpose: The 87% disk question: only current runbooks reach the LLM; the answer uses 85%. *(needs `.env` and internet; LLM wording varies: judge the meaning)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
Q="Broker 4's data volume just hit 87%. Should that have paged someone?"
./run.sh ask "$Q" 2>/dev/null | sed -n '/retrieved/,/sources offered/p'
```

Expected message: Retrieved list has `disk-capacity` but **not** `disk-capacity-2025`; the answer says it pages because the threshold is **85%**.

Pass rule: No 2025 file retrieved; answer cites 85%.

TC-19 · Same question, filter OFF (compare)

Purpose: Show the problem: the outdated 90% runbook is in the context. *(needs `.env` and internet)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
Q="Broker 4's data volume just hit 87%. Should that have paged someone?"
./run.sh ask-off "$Q" 2>/dev/null | sed -n '/retrieved/,/sources offered/p'
```

Expected message: Retrieved list includes `disk-capacity-2025` (the old 90% rule) at #2. The answer may still be right; that's luck, not design.

Pass rule: `disk-capacity-2025` is retrieved.

TC-20 · Exact value question

Purpose: An exact port number comes from the current runbook only. *(needs `.env` and internet; LLM wording varies: judge the meaning)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
Q="What port is the schema registry on?"
./run.sh ask "$Q" 2>/dev/null | sed -n '/generate/,/sources offered/p'
```

Expected message: Answer says port **8085**, never the old 8081.

Pass rule: 8085 present, 8081 absent.

TC-21 · Renamed on-call service

Purpose: A renamed PagerDuty service: the old name must not appear. *(needs `.env` and internet; LLM wording varies: judge the meaning)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
Q="Which PagerDuty service do I trigger for a production Kafka problem?"
./run.sh ask "$Q" 2>/dev/null | sed -n '/generate/,/sources offered/p'
```

Expected message: Answer names **streaming-platform-prod**, never `kafka-ops`.

Pass rule: Current name only.

TC-22 · Unanswerable question

Purpose: The corpus has no message-size limit; the system should say so, not invent one. *(needs `.env` and internet; LLM wording varies: judge the meaning)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
Q="What's the largest message size our brokers will accept?"
./run.sh ask "$Q" 2>/dev/null | sed -n '/generate/,/sources offered/p'
```

Expected message: The answer says the runbooks don't specify it. It shouldn't give a number such as 1 MB.

Pass rule: Judge by reading: a refusal or "not documented" passes, an invented number fails.

F. Negative tests

Pre-condition for this group: TC-03 done. Each test restores the data itself.

TC-23 · Broken manifest is caught

Purpose: Mark an old file current, make a version a string, break a date: the checks must fail.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && M=simple_rag/corpus/kafka/manifest.json && cp $M /tmp/b1-manifest.bak
.venv/bin/python - <<'PY'
import json; p = "simple_rag/corpus/kafka/manifest.json"; m = json.load(open(p))
m["files"]["disk-capacity-2025.md"]["superseded"] = False
m["files"]["topic-retention.md"]["version"] = "3"
m["files"]["schema-registry.md"]["effective_date"] = "2026-3-20"
json.dump(m, open(p, "w"), indent=2)
PY
.venv/bin/python simple_rag/b1_check.py --no-trap > /tmp/b1-out.txt; echo "exit=$?"
grep -E "FAIL" /tmp/b1-out.txt | head -4
cp /tmp/b1-manifest.bak $M
echo "restored: $(.venv/bin/python simple_rag/b1_check.py --no-trap | tail -1)"
```

Expected message: `exit=1`, FAIL lines naming each injected error, then `restored: PASS`.

Pass rule: Checks fail on bad data and pass again after restore.

TC-24 · Broken golden set is caught

Purpose: A case that cites the old file, and a banned string a correct answer contains.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && G=simple_rag/golden_kafka.jsonl && cp $G /tmp/b1-golden.bak
.venv/bin/python - <<'PY'
import json; p = "simple_rag/golden_kafka.jsonl"; rows = [json.loads(l) for l in open(p)]
rows[0]["expect"]["must_cite"] = [{"doc": "Kafka Broker Disk Capacity Runbook", "version": 2}]
rows[2]["expect"]["must_not_contain"].append("3 days")
open(p, "w").write("".join(json.dumps(r) + "\n" for r in rows))
PY
.venv/bin/python simple_rag/b1_check.py --no-trap > /tmp/b1-out.txt; echo "exit=$?"
tail -1 /tmp/b1-out.txt
cp /tmp/b1-golden.bak $G
echo "restored: $(.venv/bin/python simple_rag/b1_check.py --no-trap | tail -1)"
```

Expected message: `exit=1`, `FAIL (4)`, then `restored: PASS`.

Pass rule: Fails with 4 findings, passes after restore.

TC-25 · Chroma without metadata is caught

Purpose: Ingest without the manifest (like the pre-Step-8 system): the Chroma check must fail.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag && export RAG_TRACE=0
mv simple_rag/corpus/kafka/manifest.json /tmp/b1-manifest.keep
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka_nometa
.venv/bin/python simple_rag/rag.py ingest --formats md >/dev/null 2>&1
mv /tmp/b1-manifest.keep simple_rag/corpus/kafka/manifest.json
.venv/bin/python simple_rag/b1_check.py --no-trap --chroma > /tmp/b1-out.txt 2>&1
echo "exit=$?"
grep -m1 "no \['doc_title'" /tmp/b1-out.txt; tail -1 /tmp/b1-out.txt
.venv/bin/python - <<'PY'
import chromadb
chromadb.PersistentClient(path="simple_rag/.chroma").delete_collection("kafka_nometa")
print("test collection removed")
PY
export RAG_COLLECTION=kafka
```

Expected message: `exit=1`, `... no ['doc_title', 'version', 'effective_date', 'superseded'] in Chroma metadata ...`, `FAIL (18)`, `test collection removed`.

Pass rule: The check refuses a store without version metadata.

G. Scenarios

Pre-condition for this group: TC-07 done. Each scenario restores the published corpus at the end.

TC-26 · The filter trusts the metadata (wrong flag)

Purpose: If someone flips the flags by mistake, the filter serves the wrong file. This proves the manifest must be maintained.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
.venv/bin/python - <<'PY'
import json; p = "simple_rag/corpus/kafka/manifest.json"; m = json.load(open(p))
m["files"]["disk-capacity.md"]["superseded"] = True
m["files"]["disk-capacity-2025.md"]["superseded"] = False
json.dump(m, open(p, "w"), indent=2)
PY
./run.sh ingest >/dev/null 2>&1
echo "== flags swapped =="
./run.sh inspect 2>/dev/null | sed -n '/filter ON/,$p' | grep disk-capacity
git checkout -- simple_rag/corpus/kafka/manifest.json && ./run.sh ingest >/dev/null 2>&1
echo "== restored =="
./run.sh inspect 2>/dev/null | sed -n '/filter ON/,$p' | grep disk-capacity
```

Expected message: After the swap, filter ON returns `disk-capacity-2025.md` as current. After the restore it returns `disk-capacity.md`.

Pass rule: The swap changes what's served, and the restore brings it back.

TC-27 · Publish a new runbook version

Purpose: The documented update procedure: add v4, mark v3 superseded, re-ingest. Only v4 is served.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-rag
.venv/bin/python - <<'PY'
import json, pathlib
k = pathlib.Path("simple_rag/corpus/kafka")
t = (k / "disk-capacity.md").read_text()
t = t.replace("Version 3 · Effective 2026-05-01", "Version 4 · Effective 2026-09-01")
(k / "disk-capacity-v4.md").write_text(t.replace("**85%**", "**80%**"))
m = json.load(open(k / "manifest.json"))
m["files"]["disk-capacity.md"]["superseded"] = True
m["files"]["disk-capacity-v4.md"] = {"doc": "Kafka Broker Disk Capacity Runbook",
    "version": 4, "effective_date": "2026-09-01", "superseded": False}
json.dump(m, open(k / "manifest.json", "w"), indent=2)
PY
./run.sh ingest >/dev/null 2>&1
echo "== v4 published =="
Q="At what disk usage does a Kafka broker page someone?"
./run.sh inspect "$Q" 2>/dev/null | sed -n '/filter ON/,$p' | grep disk-capacity
rm simple_rag/corpus/kafka/disk-capacity-v4.md
git checkout -- simple_rag/corpus/kafka/manifest.json
./run.sh ingest >/dev/null 2>&1 && echo "== restored to the published corpus =="
```

Expected message: Filter ON returns `disk-capacity-v4.md` (current) and no other disk-capacity file; then `== restored to the published corpus ==`.

Pass rule: Only v4 is served while it's current.

H. Project branch

Pre-condition for this group: TC-02 done. TC-30 also needs TC-03 (it reuses that Python environment).

TC-28 · The golden set was frozen before any code change

Purpose: Git history proves the tests existed before the fix.

Commands (copy all, paste, Enter):

```
cd ~/b1-test
R=https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware.git
git clone -q -b track-b-kafka-version-aware $R b1-project && cd b1-project
git log --format='%h %ad %s' --date=short -- simple_rag/golden_kafka.jsonl
git log --format='%h %s' 6b918c2..HEAD -- simple_rag/rag.py | tail -1
git merge-base --is-ancestor 2cff45a 8c97952 \
  && echo "freeze 2cff45a comes before the first rag.py change 8c97952: True"
```

Expected message: One golden-set commit `2cff45a ... Freeze B1 golden set ...`; first rag.py change `8c97952 ... step 7`; `... True`.

Pass rule: The freeze commit is an ancestor of the first rag.py change.

TC-29 · No secrets or personal email in the public repo

Purpose: The published code holds no API keys, .env files or personal email.

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-project
S=$(git grep -IE 'sk-[A-Za-z0-9]{20,}|OPENAI_API_KEY=[A-Za-z0-9-]{12,}' | wc -l | tr -d ' ')
echo "secret-like strings: $S"
echo ".env files tracked: $(git ls-files | grep -cE '(^|/)\.env$')"
echo "gmail in commit authors: $(git log --format='%ae %ce' | grep -c gmail)"
```

Expected message: `secret-like strings: 0`, `.env files tracked: 0`, `gmail in commit authors: 0`.

Pass rule: All zero.

TC-30 · Rebuild every step from scratch (optional, ~4 min)

Purpose: One script replays Steps 0–12 from a fresh clone and compares fingerprints. *(takes ~4 min)*

Commands (copy all, paste, Enter):

```
cd ~/b1-test/b1-project
export SKIP_STEP2=1 VENV=~/b1-test/b1-rag/.venv
bash b1-track-b/tools/recreate_track_b.sh ~/b1-test/rebuild 2>&1 \
  | grep -E "MATCH|AFTER:|Done"
```

Expected message: `AFTER: 0.0% superseded · 9/9 · 7/7 · 3/3`, seven `... MATCH` lines, `Done · Steps 0–12 recreated`.

Pass rule: Every fingerprint says MATCH.

Cleanup

Run this when you're finished. It deletes the test folder, including the `.env` copy made in TC-17:

```
rm -rf ~/b1-test /tmp/b1-manifest.bak /tmp/b1-golden.bak /tmp/b1-out.txt
```

4. Test record

ID	Test	Pass	Fail	Notes
TC-01	Prerequisites are installed	☐	☐	
TC-02	Clone the deployment branch from GitHub	☐	☐	
TC-03	Create the Python environment	☐	☐	
TC-04	Corpus, manifest and golden set are consistent	☐	☐	
TC-05	The similarity trap is real	☐	☐	
TC-06	Frozen files have the published fingerprints	☐	☐	
TC-07	Ingest the runbooks into Chroma	☐	☐	
TC-08	Every chunk in Chroma carries its version	☐	☐	
TC-09	Raw Chroma query, no project scripts	☐	☐	

ID	Test	Pass	Fail	Notes
TC-10	See the filter's effect inside Chroma	☐	☐	
TC-11	BEFORE: filter OFF	☐	☐	
TC-12	AFTER: filter ON	☐	☐	
TC-13	Case by case: what the filter fixed and broke	☐	☐	
TC-14	Results are deterministic and match the committed ones	☐	☐	
TC-15	The filter adds no meaningful latency	☐	☐	
TC-16	Ablation summary table	☐	☐	
TC-17	Provide the API key (.env)	☐	☐	
TC-18	Stale-threshold question, filter ON	☐	☐	
TC-19	Same question, filter OFF (compare)	☐	☐	
TC-20	Exact value question	☐	☐	
TC-21	Renamed on-call service	☐	☐	
TC-22	Unanswerable question	☐	☐	
TC-23	Broken manifest is caught	☐	☐	
TC-24	Broken golden set is caught	☐	☐	
TC-25	Chroma without metadata is caught	☐	☐	
TC-26	The filter trusts the metadata (wrong flag)	☐	☐	
TC-27	Publish a new runbook version	☐	☐	
TC-28	The golden set was frozen before any code change	☐	☐	
TC-29	No secrets or personal email in the public repo	☐	☐	
TC-30	Rebuild every step from scratch (optional, ~4 min)	☐	☐	

Tested by: _____ Date: _____ Result: ____ / 30 PASS

5. If a test fails

Symptom	Likely cause	Fix
`No .venv yet — run: ./run.sh setup`	TC-03 not run, or it failed	Run TC-03 and read its last lines
`collection ... is empty` / 0 chunks	TC-07 (ingest) not run	Run TC-07, then retry
TC-11/12 numbers differ	Corpus or golden set changed	Run TC-06: the fingerprints must match; re-clone if not
`ask needs .env`	TC-17 not done	Run TC-17 (or create `.env` with your own key)
TC-18 to 22 hang or time out	No internet, or the LLM proxy is down	Retry later. TC-01 to 17 and 23 to 30 don't need the LLM
TC-26 or TC-27 left the corpus modified	The block was interrupted	`cd ~/b1-test/b1-rag && git checkout -- simple_rag/corpus && rm -f simple_rag/corpus/kafka/disk-capacity-v4.md && ./run.sh ingest`

Symptom	Likely cause	Fix
`fatal: destination path ... already exists`	An earlier run left `~/b1-test` behind	Run the cleanup command, then start at TC-02